NAVIGATING THE TRADE-OFF BETWEEN LEARNING EFFICACY AND PROCESSING EFFICIENCY IN DEEP NEURAL NETWORKS

Anonymous authors
Paper under double-blind review

ABSTRACT

A number of training protocols in machine learning seek to enhance learning efficacy by training a single agent on multiple tasks in sequence. Sequential acquisition exploits the discovery of common structure between tasks in the form of shared representations to improve learning speed and generalization. The learning of shared representations, however, is known to impair the execution of multiple tasks in parallel. The parallel execution of tasks results in higher efficiency of processing and is promoted by separating representations between tasks to avoid processing interference. Here, we build on previous work involving shallow networks and simple task settings suggesting that there is a trade-off between learning efficacy and processing efficiency, mediated by the use of shared versus separated representations. We show that the same tension arises in deep networks and discuss a meta-learning algorithm for an agent to manage this trade-off in an unfamiliar environment. We display through different experiments that the agent is able to successfully optimize its training strategy as a function of the environment.

1 INTRODUCTION

Many recent advances in machine learning can be attributed to the ability of neural networks to exploit the benefits of shared representations for learning. In that field, "multi-task learning" (as distinct from "multitasking" in cognitive psychology)[1] refers to settings in which an agent is trained on a set of auxiliary tasks that share representations with the task to be learned, exploiting the fact that similarities among those tasks can lead to shared representations that can help improve generalization and thereby acquisition of the target task (Baxter, 1995; Caruana, 1997; Maurer et al., 2016). This stands in contrast to the ability of a network architecture to carry out multiple processes independently at the same time as it is heavily used, for example, in computing clusters to distribute independent units of computation in order to minimize compute time. The processing efficiency associated with parallel processing, however, is supported by a separation of task representations between tasks in artificial networks (Alon et al., 2017; Musslick et al., 2016; Musslick & Cohen, J. D., 2019). This is also evidenced by neuroimaging studies, showing that higher amounts of neural pattern separation between tasks predict better parallel processing performance in humans (Garner & Dux, 2015; Nijboer et al., 2014).

Recent work has hypothesized that the trade-off between shared and separated representations is critical to certain aspects of human cognition (Musslick et al., 2016; 2017). Specifically, while the sharing of representation across tasks allows for quicker learning and greater generalization, it poses the risk of cross-talk, thus limiting the number of tasks that can be executed in parallel (i.e. processing efficacy). Navigation of this trade-off by the human brain may explain why we are able to parallelize some tasks in daily life (such as talking while walking) but not others (for example, doing two mental arithmetic problems at the same time). Musslick et al. (2017) have shown that this trade-off is also faced by artificial neural networks when trained to perform simple synthetic tasks. This

[1] In other research domains, such as cognitive neuroscience or cognitive psychology, multitasking refers to the simultaneous *execution* of multiple tasks, whereas in machine learning, multitasking often refers to the simultaneous *acquisition* of multiple tasks.

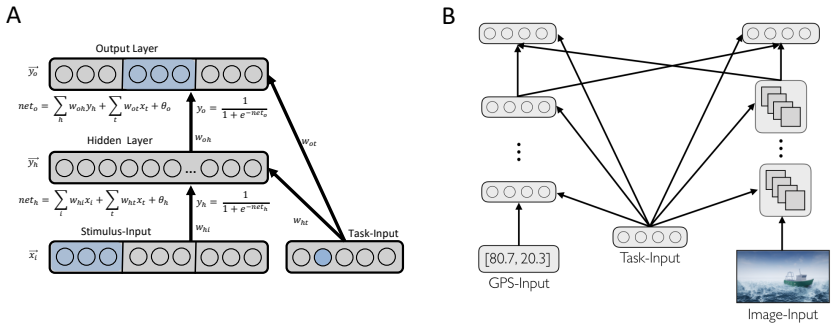


Figure 1: Neural network architectures considered in previous and current work. (A) Neural network architecture from Musslick et al. (2016). (B) Adapted neural architecture used for the simulation experiments described in the main text.

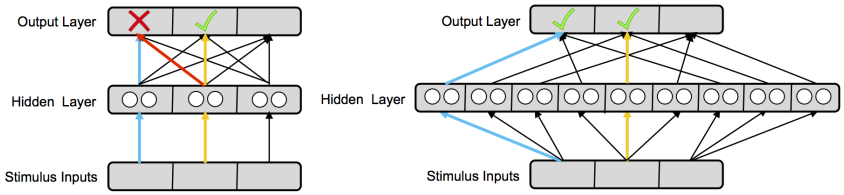


Figure 2: Network structure for minimal basis set (left) and tensor product (right) representations and the effects of multitasking in each. Red cross indicates error in execution of task because of interference whereas green check-mark indicates successful execution.

previous work demonstrates both computationally and analytically that the improvement in learning speed through the use of shared representation comes at the cost of limitations in parallel processing.

While these studies were informative, they were limited to shallow networks and simple tasks. Moreover, this work raises an important, but as yet unanswered question: how can an agent optimally trade-off the efficiency of learning against processing efficiency? In this work, we: (a) show that this trade-off also arises in deep convolutional networks used to learn more complex tasks; (b) demonstrate that this trade-off can be managed by using serial vs parallel task execution and training to control whether or not representations are shared; (c) propose and evaluate a meta-learning algorithm that can be used by a network to regulate its training and optimally manage the trade-off between learning efficacy and processing efficiency in an environment with unknown serialization costs.

2 Background

2.1 Definition of Tasks and Parallel Processing

Consider an environment in which there are multiple stimulus input dimensions (e.g. corresponding to different sensory modalities) and multiple output dimensions (corresponding to different response modalities). Given an input dimension I (e.g. an image) and an output dimension O (e.g. object category) of responses, a task $T : I \rightarrow O$ represents a mapping between the two (e.g. mapping a set of images to a set of object categories), such that the mapping is independent of any other. Thus, given N different input dimensions and K possible output dimensions, there is a total of NK possible independent tasks that the network can learn to perform. Finally, parallel processing refers to the simultaneous execution of multiple tasks, i.e. within one forward-pass from the inputs to the outputs of a network. Note that such parallel processing differs from multi-task learning protocols in that the parallel execution of tasks considered here requires tasks to map different input dimensions to different output dimensions in a way that each is independent of the other (Lesnick et al., 2020; Pashler, 1994), whereas typically in multi-task learning all tasks map the same input dimension to different output dimensions (Caruana, 1997).

2.2 PROCESSING SINGLE AND MULTIPLE TASKS BASED ON TASK PROJECTIONS

We focus on a network architecture that has been used extensively in previous work (Cohen et al., 1990; Botvinick et al., 2001; Musslick et al., 2017) (shown in Figure 1A). Here, in addition to the set of stimulus inputs, there is also an input dimension to indicate which task the network should perform. This task vector is projected to the hidden units and output units using learned weights. The hidden unit task projection biases the hidden layer to calculate a specific representation required for each task, whereas the output unit projection biases the outputs to only allow the output that is relevant for the task. The functional role of the task layer is inspired by the notion of cognitive control and attention in psychology and neuroscience, that is, the ability to flexibly guide information processing according to current task goals (Shiffrin & Schneider, 1977; Cohen, 2017). Assuming that the task representations used to specify different tasks are orthogonal to one another (e.g., using a one hot code for each), then parallel processing can be specified by a superposition (sum) of the representations for the desired tasks in the task input layer. The weights learned for the projections from the task input units to units in the hidden layers, together with those learned within the rest of the network, co-determine what type of representation (shared or separate) the network uses.

2.3 MINIMAL BASIS SET VS TENSOR PRODUCT REPRESENTATIONS

Previous work (Feng et al., 2014; Musslick et al., 2016; 2017) has established that, in the extreme, there are two ways that different tasks can be represented in the hidden layer of a two-layer network. The first representational scheme is the *minimal basis set* (shown on the left in Figure 2), in which all tasks that rely on the same input encode the input in the same set of hidden representations. The second scheme is the *tensor product* (shown on the right in Figure 2), in which the input for each task is separately encoded in its own set of hidden representations. Thus, the minimal basis set maximally shares representations across tasks whereas the tensor product uses separate representations for each.

These two representational schemes pose a fundamental trade-off. The minimal basis set provides a more efficient encoding of the inputs, and allows for faster learning of the tasks because of the sharing of information across tasks. However, it prohibits executing more than one task at a time (i.e. any parallel processing). This is because, with the minimal basis set, attempting to execute two tasks concurrently causes the implicit execution of other tasks due to the representational sharing between tasks. In contrast, while the tensor product network scheme is less compact, parallel processing is possible since each task is encoded separately in the network, so that cross-talk does not arise among them (see Figure 2 for an example of parallel processing and its effects in both representational schemes). However, learning the tensor product representation takes longer since it cannot exploit the sharing of representations across tasks.

The type of representation learned by the network can be determined by the type of task-processing on which it is trained. Serial task training involves training on tasks one at a time and generally induces shared representations. In contrast, parallel task training involves training to execute multiple tasks concurrently and produces separate representations. This occurs because using shared representations when parallel processing causes interference and thus error in task execution. In order to minimize this error and the cross-talk that is responsible for it, the network learns task projection weights that lead to separate representations for the tasks. In serial task training, there is no such pressure, as there is no potential for interference when executing one task at a time, and so the network can use shared representations. These effects have been established both analytically and experimentally for shallow networks with one hidden-layer trained to perform simple synthetic tasks (Musslick et al., 2017; Musslick & Cohen, J. D., 2019), and analytically for deep network architectures (Alon et al., 2017). Below, we report results suggesting that they generalize to deep neural networks trained on more complex tasks.

3 META-LEARNING FOR OPTIMAL AGENT

The trade-off described above begs the following question: How can it be managed in an environment with unknown properties? That is, how does an agent decide whether to pursue serial or parallel task training in a new environment so that it can learn the tasks most efficiently while maximizing the rewards it receives?

Suppose an agent must learn how to optimize its performance on a given set of tasks in an environment over τ trials. At trial t, for each task, the agent receives a set of inputs $\mathbf{X} = \{\mathbf{x}_k\}_{k=1}^K$ and is expected to produce the correct labels $\mathbf{Y} = \{\mathbf{y}_k\}_{k=1}^K$ corresponding to the task. Assuming that each is a classification task, the agent's reward is its accuracy for the inputs, i.e. $R_t = \frac{1}{K}\sum_{k=1}^K \mathbb{1}_{\hat{\mathbf{y}}_k=\mathbf{y}_k}$ where $\hat{\mathbf{y}}_k$ is the predicted label by the agent for each input $\mathbf{x}_k$. On each trial, the agent must perform all the tasks, and it can choose to do so either serially (i.e. one task at a time) or in parallel (i.e. executing multiple tasks simultaneously). After completion of the task and observation of the rewards, the agent also receives the correct labels $\mathbf{Y}$ for the tasks in order to train itself to improve task performance. Finally, assume that the agent's performance is measured across these trials through the entire course of learning and its goal is to maximize the sum of these rewards across all tasks.

To encode the time cost of serial task execution, we assume the environment has some unknown *serialization cost* c that determines the cost of performing tasks one at a time. We assume that the reward for each task when done serially is $\frac{R_t}{1+c}$ where R_t is the reward as defined before. The serialization cost therefore discounts the reward in a multiplicative fashion for serial task execution. We assume that $0 \leq c \leq 1$ so that $c = 0$ indicates there is no cost enforced for serial performance whereas $c = 1$ indicates that the agent receives half the reward for all the tasks by performing them in sequence. Note that the training strategy the agent picks not only affects the immediate rewards it receives but also the future rewards, as it influences how effectively the agent learns the tasks to improve its performance in the future. Thus, depending on the serialization cost, the agent may receive lower reward for picking serialization but gains a benefit in learning speed that may or may not make up for it over the course of the entire learning episode. This question is at the heart of the trade-off the agent must navigate to make the optimal decision. We note that this is one simple but intuitive way to encode the cost of doing tasks serially but other mechanisms are possible.

3.1 Approximate Bayesian Agent

We assume that, on each trial, the agent has the choice between two training strategies to execute and learn the given tasks - by serial execution or parallel execution. The method we describe involves, on each trial, the agent modeling the reward dynamics under each training strategy for each task and picking the strategy that is predicted to give the highest discounted total future reward across all tasks. To model the reward progress under each strategy, we first define the reward function for each strategy, $f_{A,i}(t)$, which gives the reward for a task i under strategy A assuming strategy A has been selected t times. The reward function captures the effects of both the strategy's learning dynamics and unknown serialization cost (if it exists for the strategy). Here, $A \in \{S, P\}$ where S represents the serial processing strategy and P represents the parallel processing strategy.

We can use the reward function to get the reward for a task i at trial t' when selecting strategy A. Let $a_1, a_2, \ldots, a_{t'-1}$ be the strategies picked at each trial until trial t'. Then,

$$R_{t'}^{(A,i)} = f_{A,i}\left(\sum_{t=1}^{t'-1} \mathbb{1}_{a_t=A}\right).$$

is the reward for task i at trial t' assuming we pick strategy A.

Given the reward for each task, the agent can get the total discounted future reward for a strategy A from trial t' onward assuming we repeatedly select strategy A:

$$R_{\geq t'}^{(A)} = \sum_{t=t'}^{\tau} \mu(t)\left(\sum_{i=1}^{N} R_t^{(A,i)}\right),$$

where $\mu(t)$ is the temporal discounting function, N is the total number of tasks, and τ is the total number of trials the agent has to maximize its reward on the tasks.

We now discuss how the agent maintains its estimate of each strategy's reward function for each task. The reward function is modeled as a sigmoidal function, the parameters of which are updated on each trial. Specifically, for a strategy A and task i, using parameters $\theta_{A,i} = \{w_1, b_1, w_2, b_2\}$, we model the reward function as $f_{A,i}(t) = \sigma(w_2 \cdot \sigma(w_1 \cdot t + b_1) + b_2)$.

We place a prior over the parameters $p(\theta_{A,i})$ and compute the posterior at each trial t' over the parameters $p(\theta_{A,i}|D_{t'})$, where $D_{t'}$ is the observed rewards until trial t' using strategy A on task i. Be-

cause the exact posterior is difficult to compute, we calculate the approximate posterior $q(\theta_{A,i}|D_{t'})$ using variational inference (Wainwright et al., 2008). Specifically, we use Stein variational gradient descent (SVGD) (Liu & Wang, 2016), which is a deterministic variational inference method that approximates the posterior using a set of particles that represent samples from the approximate posterior. The benefit of using SVGD is that it allows the number of particles used to be selected so as to increase the complexity of the approximate posterior, while ensuring that the time it takes to compute this approximation is practical. Because the posterior needs to be calculated repeatedly during training of the network, we found SVGD to offer the best properties – the approximate posterior is much quicker to compute than using MCMC techniques, while allowing for a complex approximation compared to using a simple Gaussian variational approximation to the posterior.

At each trial t', the agent uses its estimate of the total discounted future reward for serialization and parallelization ($R^{(S)}_{\geq t'}$ and $R^{(P)}_{\geq t'}$, respectively) to decide which strategy to use. This can be thought of as a two-armed bandit problem, in which the agent needs to adequately explore and exploit to decide which arm, or strategy, is better. Choosing the serialization training regimen may give initial high reward (because of the learning speed benefit) but choosing parallel processing may be the better long-term strategy because it does not suffer from any serialization cost. Thompson sampling (Thompson, 1933; Chapelle & Li, 2011; Gershman, 2018) is an elegant solution to the explore-exploit problem, involving sampling from the posterior over the parameters and taking decisions greedily with respect to the sample. It provides initial exploration as the posterior variance is large at the start because of a lack of data and then turns to exploitation when the posterior is more confident due to seeing enough data. On each trial, we use Thompson sampling to pick between the training strategies by sampling from the approximate posterior over parameters for each strategy, calculating the total discounted future reward for each strategy according to the sampled parameters, and picking the strategy corresponding to the higher reward. Note that in practice we do not re-estimate the posterior in each trial (as one new reward will not change the posterior much) but instead do it periodically when enough new rewards have been observed.

4 Related Work

The most relevant work from the multi-task learning literature focuses on maximizing positive transfer across tasks while minimizing negative transfer. This list includes work on minimizing learning interference when doing multi-task training (Teh et al., 2017; Rosenbaum et al., 2017) and reducing catastrophic interference when learning tasks one after another (Rusu et al., 2016; Kirkpatrick et al., 2017). However, to our knowledge, none of these works explicitly deal with the issue of how the type of representations a network uses affects whether it can execute tasks serially or in parallel. Similarly, while there are various various lifelong learning or continual learning methods that modulate layers to balance between sharing and separating representations (Javed & White, 2019; Lopez-Paz & Ranzato, 2017; Vuorio et al., 2019), they do not focus on contrasting the benefits of serialization vs. parallelization as defined in this work.

Additionally, as mentioned previously, we build on previous work studying the trade-off of learning efficacy vs processing efficiency in artificial neural networks (Alon et al., 2017; Feng et al., 2014; Musslick et al., 2016; 2017). Additionally, our meta-learning algorithm is similar to the one proposed by Sagiv et al. (2018). However, we explicitly use the model's estimate of future rewards under each strategy to also decide how to train the network, whereas the meta-learner in Sagiv et al. (2018) was not applied to a neural network's learning dynamics. Instead, the actual learning curve for each strategy A was defined according to pre-defined synthetic function. Our algorithm is thus applied in a much more complex setting in which estimation of each strategy's future rewards directly affects how the network chooses to be trained. Furthermore, our method is fully Bayesian in the sense that we utilize uncertainty in the parameter posterior distribution to control exploration vs exploitation via Thompson sampling. In Sagiv et al. (2018) logistic regression was combined with the ϵ-greedy method to perform this trade-off, which requires hyper-parameters to control the degree of exploration. Lastly, we assume that the serialization cost is unknown and model its effects on the future reward of each strategy whereas Sagiv et al. (2018) makes the simplifying assumption that the cost is known. Modeling the effects of an unknown serialization cost on the reward makes the problem more difficult but is a necessary assumption when deploying agents that need to make such decisions in a new environment with unknown properties.

Lastly, previous work on bounded optimality (Russell & Subramanian, 1994; Lieder & Griffiths, 2017) is also relevant, as it is closely related to the idea of optimizing a series of computations given a processing cost as our proposed meta-learner does.

5 EXPERIMENTS

In this section, we evaluate experimentally the aforementioned trade-off and proposed meta-learner model to resolve it. We first start by describing the task environment we use and the set of tasks we consider. We then describe the neural network architecture used, including the specific form of the task projection layer mentioned in Section 2.2 that we use and how training occurs for serialization and parallelization. In Sections 5.3 and 5.4, we show through experiments explicitly how the trade-off arises in the task environment. Lastly, in Section 5.5, we evaluate our proposed meta-learner's ability to navigate this trade-off in the environment given that there is an unknown serialization cost.

5.1 EXPERIMENTAL SETUP

We create a synthetic task environment using AirSim (Shah et al., 2018), an open-source simulator for autonomous vehicles built on Unreal Engine[2]. We assume a drone-agent that has two stimulus-inputs: (1) a GPS-input through which it can be given location-relevant information; (2) an image-input providing it visual information (e.g. from a camera). The agent also has two outputs: (1) a location-output designating a location in the input image; (2) an object-output designating the object the agent believes is present in the input. Based on the definition of a task as a mapping from one input to one output, this give us the following four tasks that the agent can perform:

Task 1 (GPS-localization): given a GPS location, output the position in the image of that location.

Task 2 (GPS-classification): given a GPS location, output the type of object the agent expects to be in that area based on its experience.

Task 3 (Image-localization): given a visual image, output the location of the object in the image.

Task 4 (Image-classification): given a visual image, output the type of object in the image.

Using AirSim, we simulate an ocean-based environment with a set of different possible objects (such as whales, dolphins, orcas, and boats). We create training examples for the agent by randomizing the location of the agent within the environment, the type of object present in the visual input, the location and rotation of the object, and the GPS location provided to the agent. Thus, each training instance contains a set of randomized inputs and a label for each of the tasks with respect to the specific inputs. The agent can execute each task using either single-tasking (one after another) or multitasking (in which it can execute Tasks 1 and 4 together or Tasks 2 and 3 together). Note that in this setup, only two tasks at most can be performed simultaneously as we will have conflicting outputs if we attempt to multitask more than two tasks.

5.2 NEURAL NETWORK ARCHITECTURE

The GPS-input is processed using a single-layer neural network, whereas the image-input is processed using a multi-layer convolutional neural network. The encoded inputs are then mapped via fully-connected layers to each output. We allow the task input to modify each hidden, or convolutional, layer using a learned projection of the task input specific to each layer. This is related to the idea of cognitive control in psychology (Cohen et al., 1990) but also to attention mechanisms used in machine learning (Hochreiter & Schmidhuber, 1997).

More formally, the task-specific projection for the ith layer $\mathbf{c}_i$ is computed using a matrix multiplication with learned task projection matrix $\mathbf{W}_{t,i}$ and task-input $\mathbf{x}_t$, followed by a sigmoid:

$$\mathbf{c}_i = \sigma(\mathbf{W}_{t,i}\mathbf{x}_t - \beta),$$

where β is a positive constant. The subtraction by $\beta > 0$ means that task projections are by default "off" i.e. close to being 0. For a fully-connected layer, the task projection $\mathbf{c}_i$ modifies the hidden

[2]Code and data will be released in final version of paper.

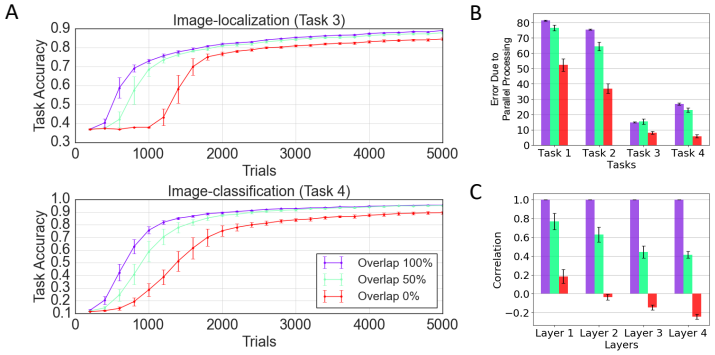


Figure 3: Effect of varying representational overlap during serial task training. Overlap varies from having completely separate representations for each task (0%, red) to completely sharing representations between tasks (100%, purple). (A) Comparison of learning speed of the networks. Accuracy for executing only one task per input pattern (training condition) increases faster for high amounts of overlap. (B) Average error for each task when executed in parallel with other tasks (testing condition; e.g. the first three bars show the error for Task 1 when executed in parallel with Task 4 for different amounts of representational sharing). Higher amounts of overlap lead to higher error in parallel processing due to interference. For 0% overlap, there is some amount of residual error due to interference because the network was never trained to execute two tasks in parallel. (C) Correlation of convolutional layer representations between Tasks 3 and Tasks 4 computed using the average representation for each layer across all the data. We show results for the tasks involving the convolutional network, as those are the more complex tasks we are interested in.

units for the ith layer $\mathbf{h}_i$ through multiplicative gating to compute the hidden units $\mathbf{h}_{i+1}$:

$$\mathbf{h}_{i+1} = g\left(\left(\mathbf{W}_{h,i}\mathbf{h}_i + \mathbf{b}_i\right) \odot \mathbf{c}_i\right),$$

where $\mathbf{W}_{h,i}$ and $\mathbf{b}_i$ are the typical weight matrix and bias for the fully-connected layer, and g is the non-linearity. For the hidden units, we let g be the rectified linear activation function (ReLU) whereas for output units it is the identity function. Similarly, for a convolutional layer the feature maps $\mathbf{h}_{i+1}$ are computed from $\mathbf{h}_i$ as:

$$\mathbf{h}_{i+1} = g\left(\left(\mathbf{h}_i * \mathbf{W}_{h,i} + \mathbf{b}_i\right) \odot \mathbf{c}_i\right),$$

where $\mathbf{W}_{h,i}$ is now the convolutional kernel. Note that we use multiplicative biasing via the task projection whereas previous work (Musslick et al., 2016; 2017) used additive biasing. We found multiplicative biasing to work better for settings in which the task projection matrix needs to be learned. A visual example of the network architecture is shown in Figure 2B.

Training in this network occurs in the typical supervised way with some modifications. To train for a specific task, we feed in the stimulus-input and associated task-input, and train the network to produce the correct label at the output associated with the task. For outputs not associated with the task, we train the network to output some default value because we did not want the network to indicate a physical response in irrelevant outputs.[3] In this work, we focus on classification-based tasks for simplicity, and so the network is trained via cross-entropy loss computed using the softmax over the network output logits and the true class label. To train the network on parallel processing, we feed in the stimulus-input and the associated task-input (indicating which set of tasks to perform concurrently) and train the network on the sum of losses computed at the outputs associated with the set of tasks. Note that we consider the localization-based tasks as classification tasks by outputting a distribution over a set of pre-determined bounding boxes that partition the image space.

[3]This is motivated by the multitasking paradigm in cognitive psychology (Pashler, 1994) where a participant is only asked to indicate a response in the relevant output modality (e.g. when asked to indicate the response with the left hand she should not also respond with the right hand). The control layer weights can be set so as to output a default value for the task not being executed, thus not impacting the learned representations.

5.3 EFFECT OF REPRESENTATION SHARING ON LEARNING EFFICACY AND PROCESSING EFFICIENCY

First, we consider the effect of the degree of shared representations on learning speed (learning efficacy) and parallel processing accuracy (processing efficiency). We control the level of sharing in the representations used by the network by manipulating the task-associated weights $\mathbf{W}_{t,i}$, which implement, in effect, the task projection for each task. The more similar the task projections are for two tasks, the higher the level of sharing because more of the same hidden units are used for the two tasks. We vary $\mathbf{W}_{t,i}$ to manipulate what percent of hidden units overlap for the tasks. Thus, 100% overlap indicates that all hidden units are used by all tasks; 50% overlap indicates that 50% of the hidden units are shared between the tasks whereas the remaining 50% are split to be used independently for each task; and 0% overlap indicates that the tasks do not share any hidden units in a layer. Note that in this experiment, during training task-associated weights are frozen based on the initialization that results in the specific overlap percentage, but the weights in the remainder of the network are free to be learned. Based on previous work (Musslick et al., 2016; 2017), we measure the degree of sharing at a certain layer between two task representations by computing the correlation between the mean representation for the tasks, where the mean is computed by averaging the activity at the layer across all training examples for a given task.

The results of the experiment manipulating the level of overlap are shown in Figure 3. These show that as overlap is increased, sharing of representations across tasks increases (as evidenced by the increase in correlations; Figure 3C), which is associated with an increase in the learning speed. That is, higher induced representational overlap between tasks can speed up learning of executing only one task at a time per input pattern (training condition; Figure 3A). However, this is associated with a degradation in parallel processing accuracy of the network (testing condition; Figure 3B), as a result of the increased interference caused by increased sharing of the representations. Note that the network with 0% overlap does not achieve error-free parallel processing performance. This suggests that there is a residual amount of interference in the network induced by single-task training that cannot be attributed do the chosen manipulation i.e. overlap between task representations.

5.4 EFFECT OF SERIAL VS PARALLEL TASK TRAINING

Having established that there is a trade-off in using shared representations in the deep neural network architecture described, we now focus on how different training regimens – using the serial vs parallel execution of tasks – impact the representations used by the network and the network's learning speed. Previous work indicated that serial task training promotes shared representations and learning efficiency (Caruana, 1997; Musslick et al., 2017) whereas training a network to execute multiple tasks in parallel yields separated representations between tasks and improvements in parallel processing performance (Musslick & Cohen, J. D., 2019). We compare different networks that vary on how much they are trained to execute tasks in parallel, from 0%, in which the network is given only serial task training, to 90%, in which the network is trained most of the time to parallelize task execution. At each time-step of training, the agent is trained on all four tasks. During serial task training, the agent is trained to perform each task one-after-another and this is repeated for all time-steps of training. Thus, the network learns to execute all four tasks one-by-one. For each task, it is given a batch[4] of inputs and the network is trained to indicate the correct output for only that task (i.e. indicate the correct response only in the output layer relevant to that task; see Section 5.2). During parallel task training, the agent is trained to execute multiple tasks in parallel at each time-step. Each batch in this training regimen contains inputs requiring the execution of exactly two tasks in parallel. That is, the network is trained to indicate a response in each of the two output layers for every input (see Sections 2.1, 5.1). A set of tasks is considered a valid multitasking condition if both tasks map different input dimensions (i.e. features from different input layers) to different output dimensions (i.e. different output layers; see Section 2.1).[5] Thus, we only included Task 1 + Task 4 and Task 2 + Task 3 in the set of parallel processing conditions from which we train on at each time-step. For both training regimens, the task-associated weights $\mathbf{W}_{t,i}$ are initialized to be uniformly high across the tasks, meaning that the network is initially biased towards using shared

[4]Note that the minibatch size of the number of inputs is fixed across both training regimens.

[5]Note that the simultaneous execution of Task 1 + Task 2 is not considered a multitasking condition because both tasks rely on the same input features (GPS input). Furthermore, Task 1 + Task 3 is not considered a valid multitasking condition because both tasks rely on the same output dimension (localization).

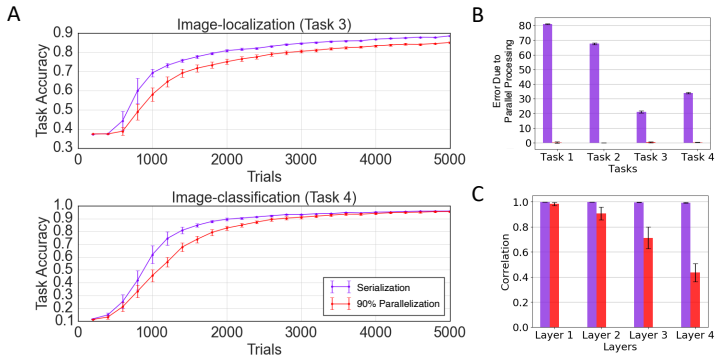


Figure 4: Effect of serial vs parallel task training. (A) Comparison of learning speed of the networks (cf. Figure 3). (B) Comparison of the error in average task performance over all data when parallelizing compared to serializing (the lack of a bar indicates no error). (C) Correlation of convolutional layer representations between Tasks 3 and Tasks 4 computed using the average representation for each layer across all the data. We show results for the tasks involving the convolutional network.

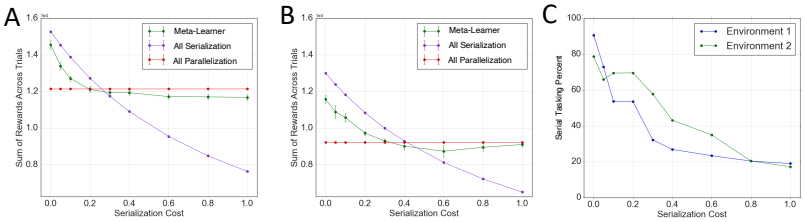


Figure 5: Evaluation of meta-learning algorithm. (A) Comparison of all methods on trade-off induced in original environment. (B) Comparison of all methods on trade-off induced in environment where noise is added to inputs. (C) Percent of trials for which meta-learner picks to do single-tasking in both environments.

representations, and all the weights (including task weights) are then learned based on the training regimen encountered by the network. We also conduct an experiment in which the network isn't as biased towards using shared representations by initializing smaller task-associated weights (see Supplementary Material). The number of examples and the sequence of examples for each task are the same for both training regimens (serialization or parallelization). The only difference is that in the case of serial task training each task is learned independently using different forward and backward passes whereas in parallelization, multiple tasks can be processed together.

The results of this experiment (Figure 4) show that as the network is trained to do more parallel processing, the learning speed of the network decreases (Figure 4A) and the correlation of the task representations also decreases (Figure 4C). Because the network is initialized to use highly shared representations, we see that a parallelization training regimen clearly forces the network to move away from this initial starting point. The effect is stronger in the later layers, possibly because these layers may contribute more directly to the interference caused when parallel processing.

5.5 Meta-Learning

Finally, having established the trade-off between serial and parallel task training, we evaluate the meta-learning algorithm to test its effectiveness in optimizing this trade-off. In order to test this in an environment with unknown serialization cost, we compare it with the extremes of always picking serial or parallel task training. We fix the total number of trials to be $\tau = 5000$ and evaluate each of the methods on varying serialization costs. For the meta-learner, we average the performances over 15 different runs in order to account for the randomness involved in its sampling choices and measure its confidence interval. We fix the order in which data is presented for the tasks for all options when comparing them. Note that the meta-learner does not know the serialization cost and so has to model its effects as part of the received reward. We create two different environments to induce different

trade-offs for rewards between serial and parallel processing. The first is a deterministic environment whereas in the second we add noise to the inputs. Adding noise to the inputs makes the tasks harder and seems to give bigger benefit to the minimal basis set (and serial task training). We hypothesize that this is the case because sharing information across tasks becomes more valuable when noisy information is provided for each task.

Figures 5A and 5B show that the meta-learning algorithm achieves a reward rate that closely approximates the one achieved by the strategy that yields the greatest reward for a given serialization cost. Additionally, note that in the extremes of the serialization cost, the meta-learner seems better at converging to the correct training strategy, while it achieves a lower reward when the optimal strategy is harder to assess. This difference is even clearer when we study the average percent of trials for which the meta-learner picks serial task training as a function of the serialization cost in Figure 5C. We see that the meta-learning algorithm is well-behaved, in that as the serialization cost increases, the percent of trials in which it selects to do single-tasking smoothly decreases. Additionally, at the points at which the optimal strategy is harder to determine, the meta-learner achieves reward closer to the worst strategy because it needs more time to sample each strategy before settling on one.

6 Discussion

In this work we study the trade-off between using shared vs separated representations in deep neural networks. We experimentally show that using shared representations leads to faster learning but at the cost of degraded parallel processing performance[6]. We additionally propose and evaluate a meta-learning algorithm to decide which training strategy is best to use in an environment with unknown serialization cost.

We believe simultaneous task execution as considered here could be important for real-world applications as it minimizes the number of forward passes needed to execute a set of tasks. The cost of a forward pass is an important factor in embedded devices (in terms of both time and energy required) and scales badly as we consider larger task spaces and more complex networks. Thus, optimally managing the trade-off between learning speed vs parallel processing capability could be crucial for maximizing efficiency in such situations.

A promising direction for future studies involves application of this meta-learner to more complex tasks. As we add more tasks, the potential for interference increases across tasks; however, as tasks become more difficult, the minimal basis set becomes more desirable, as there is even bigger benefit to sharing representations. Furthermore, in this more complicated setting, we would also like to expand our meta-learning algorithm to decide explicitly which set of tasks should be learned so that they can be executed in a parallel fashion and which set of tasks should only be executed one at a time. This requires a more complicated model, as we have to keep track of many possible strategies in order to see what will give the most reward in the future.

References

N. Alon, D. Reichman, I. Shinkar, T. Wagner, S. Musslick, Cohen, J. D., T. Griffiths, B. Dey, and K. Özcimder. A graph-theoretic approach to multitasking. advances in neural information processing systems. In *Advances in Neural Information Processing Systems*, pp. 2097—2106. Long Beach, CA, 2017.

Jonathan Baxter. Learning internal representations. In *Proceedings of the eighth annual conference on Computational learning theory*, pp. 311–320. ACM, 1995.

Matthew M Botvinick, Todd S Braver, Deanna M Barch, Cameron S Carter, and Jonathan D Cohen. Conflict monitoring and cognitive control. *Psychological review*, 108(3):624, 2001.

Rich Caruana. Multitask learning. *Machine learning*, 28(1):41–75, 1997.

Olivier Chapelle and Lihong Li. An empirical evaluation of thompson sampling. In *Advances in neural information processing systems*, pp. 2249–2257, 2011.

[6]Note that limitations in parallel processing due to shared representations may be bypassed by executing different single tasks across multiple copies of the trained network. However, this strategy appears inefficient as it requires a higher amount of memory and computation that scales with the number of tasks to be executed.

Jonathan D Cohen. Cognitive control: core constructs and current considerations. *The Wiley handbook of cognitive control*, pp. 1–28, 2017.

Jonathan D Cohen, Kevin Dunbar, and James L McClelland. On the control of automatic processes: a parallel distributed processing account of the stroop effect. *Psychological review*, 97(3):332, 1990.

Samuel F Feng, Michael Schwemmer, Samuel J Gershman, and Jonathan D Cohen. Multitasking versus multiplexing: Toward a normative account of limitations in the simultaneous execution of control-demanding behaviors. *Cognitive, Affective, & Behavioral Neuroscience*, 14(1):129–146, 2014.

KG Garner and Paul E Dux. Training conquers multitasking costs by dividing task representations in the frontoparietal-subcortical system. *Proceedings of the National Academy of Sciences*, 112 (46):14372–14377, 2015.

Samuel J Gershman. Deconstructing the human algorithms for exploration. *Cognition*, 173:34–42, 2018.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

Khurram Javed and Martha White. Meta-learning representations for continual learning. In *Advances in Neural Information Processing Systems*, pp. 1820–1830, 2019.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.

M. Lesnick, S. Musslick, B. Dey, and J. D. Cohen. A formal framework for cognitive models of multitasking. 2020. doi: https://doi.org/10.31234/osf.io/7yzdn.

Falk Lieder and Thomas L Griffiths. Strategy selection as rational metareasoning. *Psychological Review*, 124(6):762, 2017.

Qiang Liu and Dilin Wang. Stein variational gradient descent: A general purpose bayesian inference algorithm. In *Advances In Neural Information Processing Systems*, pp. 2378–2386, 2016.

David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in neural information processing systems*, pp. 6467–6476, 2017.

Andreas Maurer, Massimiliano Pontil, and Bernardino Romera-Paredes. The benefit of multitask representation learning. *The Journal of Machine Learning Research*, 17(1):2853–2884, 2016.

S. Musslick and Cohen, J. D. A mechanistic account of constraints on control-dependent processing: Shared representation, conflict and persistence. In *Proceedings of the 41st Annual Meeting of the Cognitive Science Society*, pp. 849—855, Montreal, CA, 2019.

S. Musslick, B. Dey, K. Özcimder, M. Patwary, T. L. Willke, and J. D. Cohen. Controlled vs. automatic processing: A graph-theoretic approach to the analysis of serial vs. parallel processing in neural network architectures. In *Proceedings of the 38th Annual Meeting of the Cognitive Science Society*, pp. 1547—1552, Philadelphia, PA, 2016.

S. Musslick, A. Saxe, K. Özcimder, B. Dey, G. Henselman, and J. D. Cohen. Multitasking capability versus learning efficiency in neural network architectures. In *Proceedings of the 39th Annual Meeting of the Cognitive Science Society*, pp. 829—834, London, UK, 2017.

Menno Nijboer, Jelmer Borst, Hedderik van Rijn, and Niels Taatgen. Single-task fmri overlap predicts concurrent multitasking interference. *NeuroImage*, 100:60–74, 2014.

Harold Pashler. Dual-task interference in simple tasks: data and theory. *Psychological bulletin*, 116 (2):220, 1994.

Clemens Rosenbaum, Tim Klinger, and Matthew Riemer. Routing networks: Adaptive selection of non-linear functions for multi-task learning. *arXiv preprint arXiv:1711.01239*, 2017.

Stuart J Russell and Devika Subramanian. Provably bounded-optimal agents. *Journal of Artificial Intelligence Research*, 2:575–609, 1994.

Andrei A Rusu, Neil C Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.

Y. Sagiv, S. Musslick, Y. Niv, and J. D. Cohen. Efficiency of learning vs. processing: Towards a normative theory of multitasking. In *Proceedings of the 40th Annual Meeting of the Cognitive Science Society*, pp. 1004—1009, Madison, WI, 2018.

Shital Shah, Debadeepta Dey, Chris Lovett, and Ashish Kapoor. Airsim: High-fidelity visual and physical simulation for autonomous vehicles. In *Field and service robotics*, pp. 621–635. Springer, 2018.

Richard M Shiffrin and Walter Schneider. Controlled and automatic human information processing: II. Perceptual learning, automatic attending and a general theory. *Psychol. Rev.*, 84(2):127, 1977.

Yee Teh, Victor Bapst, Wojciech M Czarnecki, John Quan, James Kirkpatrick, Raia Hadsell, Nicolas Heess, and Razvan Pascanu. Distral: Robust multitask reinforcement learning. In *Advances in Neural Information Processing Systems*, pp. 4496–4506, 2017.

William R Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3/4):285–294, 1933.

Risto Vuorio, Shao-Hua Sun, Hexiang Hu, and Joseph J Lim. Multimodal model-agnostic meta-learning via task-aware modulation. In *Advances in Neural Information Processing Systems*, pp. 1–12, 2019.

Martin J Wainwright, Michael I Jordan, et al. Graphical models, exponential families, and variational inference. *Foundations and Trends® in Machine Learning*, 1(1–2):1–305, 2008.